Relatório — Ir Além 1: Comunicação automatizada com REST e e-mail

CardioIA — Fase 3 · Victor Copque dos Reis (RM566821) · Victor Hugo Ferreira Rolim (RM568006)

1. O que foi construído

Um sistema de monitoramento em Python com três peças: uma **API REST** (`api.py`, Flask + SQLite) que recebe os sinais vitais, um **cliente REST** (`cliente.py`, biblioteca `requests`) que faz o papel do vestível e do posto de enfermagem, e uma **automação de e-mail** (`notificador.py`) que avisa a equipe quando surge um risco. A lógica clínica fica isolada em `risco.py`.

Etapa	Quem faz	Protocolo
1. Envia a leitura	`cliente.py` (vestível simulado)	HTTP `POST /api/v1/leituras`, JSON
2. Valida e guarda	`api.py`	SQLite
3. Verifica o risco	`risco.py`	—
4. Dispara o e-mail	`notificador.py`	SMTP (`smtplib`) ou arquivo `.eml`
5. Consulta	`cliente.py` (enfermagem)	HTTP `GET /leituras`, `/alertas`, `/pacientes/<id>/status`

2. Fluxo de uma leitura

1. O cliente envia o JSON da leitura (`paciente`, `seq`, `ts`, `temp`, `umid`, `bpm`, `acel`), o mesmo formato que o ESP32 publica por MQTT na Parte 2, com o cabeçalho `X-API-Key`.
2. A API confere a chave (**401** se faltar) e valida tipo e faixa de cada campo (**400** com a lista de erros, por exemplo `bpm: tipo inválido`).
3. Se o par `paciente + ts + seq` já existe, a API responde **200** e não grava de novo. Isso torna o reenvio seguro, como a deduplicação do Node-RED.
4. A API busca as últimas leituras do paciente e chama `risco.avaliar()`.
5. A leitura é gravada com o movimento, os motivos e a severidade, e a API responde **201** com a leitura classificada e o resultado do e-mail.

3. Lógica de verificação de risco

Motivo	Regra	Severidade
taquicardia	BPM > 120 em repouso; > 150 em atividade	atenção
febre	temperatura ≥ 37,8 °C	atenção
bradicardia	0 < BPM < 50	crítico
hipotermia	temperatura ≤ 35 °C	crítico
queda	aceleração ≥ 2,5 g	crítico
ausência de movimento	10 leituras seguidas a menos de 0,02 g de 1 g	crítico

Os limiares são os mesmos do ESP32, para o dispositivo e a nuvem nunca discordarem. Duas decisões merecem destaque:

- **O limite de BPM depende do movimento.** Correndo, 140 BPM é esperado; deitado, é taquicardia. O movimento vem da aceleração da própria leitura.

- **Ausência de movimento só existe na nuvem.** Ela depende do histórico: uma leitura parada não diz nada, dez seguidas sim. Um sensor em repouso real sempre oscila um pouco; ficar cravado em 1,00 g é o padrão de quem desmaiou ou caiu e não se levantou. Na simulação a janela é de 20 s; em produção seria de minutos e cruzada com o horário de sono.

BPM 0 significa "sem leitura de pulso" e não gera alerta: falta de dado não é bradicardia.

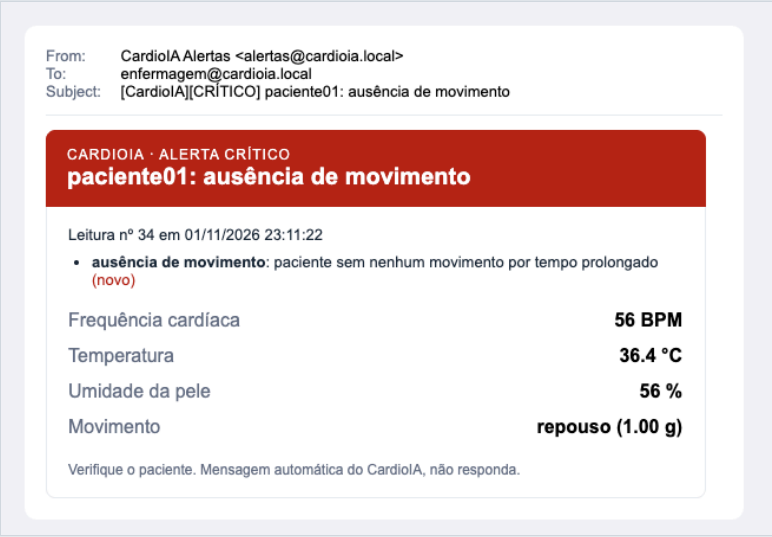
4. Automação do e-mail

O e-mail sai **somente quando aparece um motivo que a leitura anterior não tinha**. Uma taquicardia que dura dez leituras gera um e-mail, não dez; se a febre aparece no meio dela, sai um segundo, marcando a febre como "novo". Assim a enfermagem não se acostuma a ignorar as mensagens.

A mensagem tem versão em texto e em HTML: severidade no assunto (`[CardioIA][CRÍTICO] paciente01: queda`), os motivos explicados, os sinais vitais e o horário da leitura. A entrega tem dois modos, escolhidos pela variável `EMAIL_MODO` :

- `arquivo` (padrão): grava o `.eml` em `caixa_saida/` , que abre em qualquer cliente de e-mail. É a simulação pedida.
- `smtp` : envia de verdade com `smtplib` (TLS e login opcionais). Testamos com o servidor local `aiosmtpd` , que recebeu os cinco e-mails do roteiro.

Se o envio falhar, a leitura **não se perde**: ela fica gravada e o alerta é registrado com `email_status = "falhou"` , visível em `GET /alertas` para uma nova tentativa.



E-mail gerado para o alerta de ausência de movimento

5. Cliente e resiliência

O cliente usa uma `requests.Session` com timeout, a chave no cabeçalho e novas tentativas automáticas com espera crescente para erros 502, 503 e 504. Se a API estiver fora do ar, a leitura vai para uma **fila local** e é reenviada em ordem na próxima chamada, a mesma ideia de *store-and-forward* da Parte 1.

O comando `python cliente.py simular` percorre um roteiro de 40 leituras (normal, taquicardia em repouso, corrida, febre, queda, paciente imóvel, bradicardia) e no fim consulta a API com `GET` . Resultado: **cinco e-mails**, um por episódio (taquicardia, febre, queda, ausência de movimento e bradicardia), e nenhum durante a corrida. O print da execução está em `prints/01-simulacao-cliente.png` .

6. Testes e limitações

São 24 testes automatizados (`fase3/tests/test_ir_alem1_rest.py`): cada regra de risco, autenticação, validação, deduplicação, a política de e-mail, falha no envio, os dois modos de entrega e o cliente contra a API rodando de verdade, incluindo a fila offline.

É uma simulação: a chave de API é fixa, o servidor é o de desenvolvimento do Flask e não há HTTPS. Em produção seriam necessários HTTPS, autenticação por dispositivo (OAuth2 ou certificado), um servidor WSGI, um provedor de e-mail transacional e o tratamento de dado de saúde como dado sensível (LGPD, art. 11).